UI Patterns

268 terms · name, then the phrase to paste · Workbook Library

Terminal & TUI

Spinner a braille-dot spinner beside the current step's label, replaced by a green ✓ or red × when the step finishes

Progress bar a determinate progress bar with percent, transfer rate and ETA, redrawn in place on one line

Task checklist a live task checklist: one line per step with pending ○, running spinner, done ✓ and failed × states, nested sub-steps indented

Semantic color semantic color only: green success, yellow warning, red error, dim secondary text; respect NO_COLOR and plain output when piped

Aligned table list output as an aligned table with an uppercase dim header row, status column colored, no borders so it pipes cleanly

Helpful errors errors as diagnostics: a short red headline, the offending line with a caret underline, and a "help:" line with the likely fix

Interactive prompts interactive prompts in the style of @clack/prompts: arrow-key select with a ❯ cursor, answered questions collapse to one line, every answer also settable by a flag

Help & subcommands git-style subcommands (tool <noun> <verb>), --help on every command that leads with usage and 2–3 real examples, commands grouped by purpose

Machine-readable output human-friendly output by default, plus --json for stable machine-readable output and --quiet for scripts; detect when stdout is not a TTY

Tree output show the hierarchy as a tree with ├── and └── branches, directories in bold blue, sizes dimmed on the right

Diff view show changes as a colored unified diff: red – removed lines, green + added lines, dim context lines and line numbers

ASCII banner a small ASCII-art wordmark banner on first run only, in one brand color, followed by a one-line tagline

Log stream multiplexed log stream: each line prefixed with a fixed-width colored service name and dim timestamp, errors highlighted

TTY-aware output TTY-aware output: color, spinners and in-place redraws only when stdout is a terminal (isatty); plain line-by-line text with no ANSI codes when piped, when NO_COLOR is set, when TERM=dumb, or with --plain / --no-color

stdout, stderr and exit codes write the primary result to stdout and all progress, warnings and errors to stderr; exit 0 on success, 1 on failure and 2 on bad usage, and document the exit codes in --help

Dry run and confirm a --dry-run flag that prints exactly what would change and changes nothing; destructive commands ask for confirmation on a TTY, the most dangerous ones require typing the resource name, --yes / --force skips the prompt, and --no-input makes it fail instead of hanging

Suggest the next step suggest the closest command on typos ("Did you mean deploy?", using edit distance) without running it automatically, and after each successful command print a dim "Next:" line with the most likely follow-up command

Pager for long output page long output through $PAGER (default less -FIRX, which quits if it fits on one screen and keeps colors) only when stdout is a TTY, with --no-pager to turn it off

Rendered markdown render markdown output with Glamour (Go) or Rich Markdown (Python): styled headings, bullets, syntax-highlighted code blocks, word wrap to the terminal width, and an auto dark/light style

Split panes full-screen TUI with tiled bordered panes, the focused pane's border highlighted, Tab to cycle focus

List and detail list-and-detail layout: scrollable list on the left with a highlighted cursor row, the selected item's details on the right, updating as the cursor moves

Status line a bottom status line with colored segments: mode on the left, current file/item in the middle, position and background tasks on the right

Key hint footer a dim footer showing the 4–6 most useful key bindings for the current view, with ? toggling a full help overlay

Terminal dashboard a live terminal dashboard: labeled bar meters for CPU/memory, a braille line graph of history, and a sorted process table below

Fuzzy finder a fuzzy finder: query line at the bottom, results above with matched characters highlighted, a preview pane of the selected item

Popup dialog a centered popup dialog with a double-line border over dimmed content, y/n buttons with the safe choice focused by default

Tabs a tab strip across the top with numbered views (1 Overview, 2 Logs, 3 Config), the active tab inverted, number keys to switch

File tree a collapsible file tree sidebar with ▸/▾ toggles, the cursor row highlighted, git status markers beside changed files

Inline TUI render the interactive UI inline below the prompt (not alternate-screen full-screen), so the final result stays in the terminal scrollback

Vim keybindings keyboard-first navigation with vim keys (h/j/k/l, g/G, / search, : command mode) and arrow-key equivalents

Terminal form a terminal form built with Charm Huh: a group of fields (text Input, Select, Confirm) with a thick left border on the focused field, validation shown inline, tab / shift+tab to move, and an accessible plain mode; use gum for the same in shell scripts

Gauges and charts charts drawn in the terminal with Ratatui widgets: Gauge for a percentage with a centered label, LineGauge for a thin meter, Sparkline for recent values, BarChart with labels, and Chart or Canvas with the braille marker for line plots

Coding-agent TUI a coding-agent TUI like Claude Code or Crush: scrolling transcript with tool calls as collapsible ● lines, edits as inline colored diffs, a bordered permission prompt (Yes / Yes, always / No) before running commands, a fixed rounded input box at the bottom, and a dim status line with model, token count and mode

Box-drawing borders boxes drawn with rounded box-drawing borders (┌─┐ └─┘), double lines only for warnings

Charm style Charm-style TUI built with Bubble Tea, styled with Lip Gloss (rounded borders, generous padding, hot-pink and violet accents, soft gray secondary text), using Bubbles components, Huh for forms and Glamour for markdown

Dense & utilitarian information-dense utilitarian layout: no wasted rows, minimal borders, many columns of numbers, color only for thresholds

Retro phosphor retro phosphor look: monochrome green on black with a soft glow and faint scanlines, like a VT100

Powerline prompt a powerline-style segmented prompt: colored blocks for directory, git branch and status joined by arrow separators, Nerd Font icons

Named color themes use the Catppuccin Mocha palette (or: Dracula / Nord / Gruvbox / Tokyo Night) for all UI colors

Web

Hero section a centered hero section: small announcement pill, a 2-line headline (text-5xl to text-6xl, tight tracking), one sentence of subtext, a primary and a secondary button, and a product screenshot below that bleeds off the fold

Bento grid a bento grid of features: CSS grid with 4 columns and mixed tile spans (one 2x2 hero tile, a tall tile, wide tiles), rounded-2xl tiles with a subtle border, each tile one feature with a small live visual

App shell with sidebar an app shell: fixed 240px left sidebar with workspace switcher, icon + label nav items and the active item highlighted, a slim top bar with search and avatar menu, and a scrollable main content area; sidebar collapses to icons on narrow screens

Dashboard a dashboard: a row of 4 KPI cards (label, big number, colored delta vs last period), one wide line chart with a date-range picker, a smaller breakdown chart beside it, and a recent-activity table below

Docs layout a three-column docs layout: left sidebar page tree with collapsible sections, center article max-w-prose with headings, code blocks and callouts, right "On this page" TOC that highlights the current heading on scroll (scroll-spy), search with ⌘K in the header

Pricing table a 3-tier pricing section: monthly/yearly toggle with a "save 20%" badge, three cards with plan name, big price per month, one-line audience, ✓ feature list and a button; the middle plan outlined in the accent color with a "Most popular" badge

Split screen a split-screen layout: left half a full-height brand panel with a gradient or image and a customer quote, right half a centered sign-in form (max-w-sm) with email, password, primary button and "Continue with Google"; stack to a single column under md

Masonry grid a masonry grid: 4 equal columns (CSS columns-4 or a JS masonry layout), cards keep their natural image aspect ratio, consistent gutters, caption on hover, infinite load at the bottom; 2 columns on mobile

Long-form article a long-form article layout: max-w-[65ch] centered column, serif or high-quality sans body at 18–20px with 1.6 line height, large headline, byline with avatar and reading time, a drop cap, pull quotes, and images that break out wider than the text; thin reading-progress bar at the top

Alternating feature rows alternating feature rows: each row a two-column section (screenshot + text with eyebrow, heading, 2 lines and a "Learn more →" link), image side flips every row, generous vertical spacing; stacks image-first on mobile

Sticky header a sticky header (position: sticky; top: 0) with a translucent background and backdrop-blur, a 1px bottom border and slight shadow that appear only after the page scrolls; keep it short, about 56–64px

Mega menu a mega menu on the "Products" nav item: a full-width panel with 3 labeled columns of links (icon, title, one-line description) plus a featured card on the right; opens on hover with a short delay and on click, closes on Escape (Radix NavigationMenu)

Hamburger and off-canvas menu below md, collapse the nav into a ☰ button that opens a left off-canvas drawer (shadcn/ui Sheet) with large tap targets, the current page highlighted, a primary button at the bottom; dim the page behind, close on ✕, outside click or Escape, and trap focus while open

Breadcrumbs breadcrumbs above the page title (nav aria-label="Breadcrumb"): ancestors as muted links separated by ›, the current page in bold and not a link (aria-current="page"), middle items collapsing into a … dropdown when the trail is long

Tabs underline tabs under the page title (Radix Tabs / shadcn/ui Tabs): active tab with a 2px accent underline and bold label, arrow keys move between tabs, the selected tab is reflected in the URL

Pagination, load more, or infinite scroll pick by task: numbered pagination for results and tables (‹ Previous, first page, pages around the current one, …, last page, Next ›, current page in the URL as ?page=3); a "Load more" button as the middle option for catalogs and lists with a footer (appends the next batch, shows "Showing 40 of 212", keeps scroll position and moves focus to the first new item); infinite scroll only for feeds (IntersectionObserver on a sentinel, skeleton rows while loading, restore scroll position on back)

Fat footer a fat footer: brand column with logo, one-line description and newsletter email field, then 4 link columns (Product, Company, Resources, Legal) with small uppercase headings, and a bottom bar with © year, theme/language switcher and social icons

Command palette a ⌘K / Ctrl+K command palette (cmdk / shadcn/ui CommandDialog): centered dialog near the top with a search input, grouped results (Suggestions, Navigation, Actions) with icons and shortcut hints on the right, fuzzy matching, arrow-key selection and Enter to run

Data table a data table (TanStack Table with shadcn/ui): filter input and faceted filter buttons above, sortable headers with ↑ ↓ indicators, row selection checkboxes with a bulk-action bar, colored status badges, a ⋯ row-actions menu, sticky header, and pagination with "N of M selected"

Toast toasts with Sonner: bottom-right stack that expands on hover, success/error/info variants with icons, an Undo action on destructive operations, auto-dismiss after ~5s (paused on hover), announced with role="status"

Modal dialog a modal dialog using the native <dialog> element opened with showModal() (top layer, styled ::backdrop, inert page and Escape for free), or Radix AlertDialog in React: dimmed backdrop, centered panel max-w-md with title, one-sentence description, and right-aligned Cancel + red destructive button; focus trapped, Escape and backdrop click close it, focus returns to the trigger

Popover and dropdown menu a dropdown menu (Radix DropdownMenu / shadcn/ui) anchored to a ⋯ button: items with leading icons and right-aligned shortcuts, separators between groups, a submenu with ›, the destructive item in red at the bottom; full keyboard support and collision-aware positioning; for simple popovers use the native popover attribute with popovertarget (light dismiss and Escape built in) and CSS anchor positioning (position-anchor, position-area) instead of Floating UI where supported

Form with inline validation a form with inline validation (React Hook Form + Zod): labels above inputs, validate on blur and then on change, red border and a specific fix-it message under the invalid field (aria-describedby), a green check on valid fields, a password strength meter, submit disabled while submitting with a spinner

Combobox / autocomplete a combobox (WAI-ARIA combobox pattern via Headless UI Combobox or shadcn/ui Command in a Popover): typing filters the list, matched letters bolded, arrow keys move the active option, Enter selects, a ✓ beside the current value, and a "Create …" option when nothing matches

Accordion an accordion (Radix Accordion / shadcn/ui, or native <details>/<summary>): full-width rows with a question and a chevron that rotates on open, smooth height animation, single item open at a time, buttons with aria-expanded

Kanban board a kanban board: horizontally scrolling columns with title and count, cards with title, label chips and assignee avatar, drag and drop between and within columns (dnd-kit) with a lifted, tilted card while dragging and a drop placeholder, keyboard dragging supported

Stepper / wizard a multi-step wizard: horizontal stepper with numbered circles (✓ for completed, ring for current, gray for upcoming) joined by a progress line, one step's fields per screen, Back and Continue buttons, "Step 3 of 4" label, progress kept in the URL so refresh does not lose it

Sheet and drawer a responsive sheet: on desktop a right side sheet (shadcn/ui Sheet, about 400 to 480px) with title, scrollable body and a sticky footer with Cancel and Save, page dimmed but visible; under the md breakpoint switch to a Vaul-style bottom drawer with a grab handle, drag to dismiss with velocity, snap points, and the page behind scaled to about 0.95 with rounded corners; focus trapped, Escape closes

Resizable panels resizable panels (react-resizable-panels / shadcn/ui Resizable): a horizontal PanelGroup with sidebar, editor and preview, thin 1px dividers with a wider invisible hit area and a small grip, col-resize cursor and accent highlight while dragging, min and max sizes, double-click to reset, collapsible sidebar, sizes saved to localStorage (autoSaveId), and arrow keys move a focused divider

Date picker and range calendar a date range picker (react-day-picker in a shadcn/ui Popover, or React Aria DateRangePicker): a trigger field showing "Mar 12 – Mar 18, 2026" with a calendar icon, a popover with presets on the left (Today, Last 7 days, Last 30 days, This month), two months side by side, start and end days as filled circles with the days between shaded, today marked, disabled past dates, arrow-key navigation, Cancel and Apply in the footer; the text field also accepts typed dates

File upload dropzone with progress a file upload dropzone (react-dropzone): a dashed rounded area with an upload icon, "Drag files here or browse", and the accepted types and max size, highlighted in the accent color while dragging over; below it a list of files with icon, name, size, a per-file progress bar and percent, cancel (✕) while uploading, a check when done, and an inline error with Retry for failed or too-large files; the browse link is a real button for keyboard users

One-time code input a one-time code input (input-otp / shadcn/ui InputOTP): six digit slots in two groups of three, one real <input> underneath with inputmode="numeric" and autocomplete="one-time-code" so iOS and Android offer the code and pasting fills every slot, the active slot ringed with a blinking caret, auto-submit on the sixth digit, a "Resend code in 0:24" countdown, and a clear error state that shakes and clears the slots

Tooltip vs. hover card tooltips (Radix Tooltip) on every icon-only button: short label plus keyboard shortcut, dark, small, shown on hover and focus after ~300ms, never containing interactive content; and a hover card (Radix HoverCard) on @mentions showing avatar, name, handle, one-line bio, follower count and a Follow button, opening after ~500ms and staying open while the pointer moves into it; on touch the mention just links to the profile

Stacked toasts, Sonner-style stacked toasts with Sonner: newest toast in front at the bottom-right, up to 3 visible with older ones scaled to 0.95 and 0.9 and offset upward behind it, the stack expands into a spaced list on hover or focus (expand on hover), timers pause while expanded, each toast swipe-to-dismiss with an optional action button, rich colors for success and error, aria-live polite

View transitions view transitions between list and detail: give the thumbnail and the detail hero image the same view-transition-name (e.g. photo-42) so the browser morphs one into the other, cross-fade the rest; use the View Transition API (document.startViewTransition() for SPA updates, @view-transition { navigation: auto } for multi-page), tune ::view-transition-group() duration to about 300ms, and skip it under prefers-reduced-motion

Scroll-driven animation and scrollytelling scroll-driven animation in CSS: a top reading-progress bar using animation-timeline: scroll(), sections that fade and rise into view using animation-timeline: view() with animation-range: entry 0% cover 30%; a scrollytelling section with a position: sticky chart on one side whose state changes per text step on the other (IntersectionObserver or view timelines); everything static under prefers-reduced-motion

Scroll-snap carousel a scroll-snap carousel in plain CSS: overflow-x: auto with scroll-snap-type: x mandatory, cards with scroll-snap-align: start and a fixed width so the next card peeks in, scroll-padding matching the page gutter, hidden scrollbar, ‹ › buttons that scroll by one card (scrollBy with smooth behavior) and dot indicators, keyboard and touch both work; no autoplay

Microinteractions microinteractions: a like button that scales to 1.2 and back with a small particle burst and an animated count, a toggle whose thumb slides with a spring and whose track changes color, a copy button that swaps its icon to a check and its label to "Copied" for 2 seconds; 150 to 300ms each, transform and opacity only, respect prefers-reduced-motion

Hover reveal and cursor effects hover reveal on cards: on hover the card lifts (translateY(-2px), larger shadow) and an action bar fades in over the image; a subtle radial spotlight follows the cursor via CSS variables set on pointermove; wrap hover-only styling in @media (hover: hover) and (pointer: fine), and on touch devices show the actions permanently or behind a ⋯ button, and make them reachable by keyboard with :focus-within

The Linear look the Linear look: near-black background (#08090A), Inter with tight tracking, headlines with a white-to-gray gradient (bg-clip-text), 1px borders as a white/10 gradient brighter at the top, a soft indigo radial glow behind the hero, muted gray body text and restrained indigo (#5E6AD2) accents

Stripe look the Stripe look: an animated mesh gradient (violet #A960EE, coral #FF333D, sky #90E0FF, yellow #FFCB57) in a section clipped at a skewed angle (clip-path or -skew-y-6), deep navy #0A2540 headlines in a heavy sans like Söhne or Inter, slate #425466 body, a #635BFF pill button with a › arrow, floating white UI cards with layered soft shadows

Vercel / Geist monochrome the Vercel / Geist look: pure #000 background (and #FFF in light mode), Geist Sans for text and Geist Mono for code and labels, 1px #333 borders forming a visible layout grid with small + marks at intersections, tight-tracked large headlines, white primary button with black text and a dark outlined secondary, color reserved for status

shadcn/ui look the shadcn/ui look: neutral zinc palette (bg-background white, border-zinc-200, text-zinc-950, muted text-zinc-500), Geist or Inter at text-sm, cards with border and shadow-sm, rounded-md inputs and buttons with h-9, a zinc-900 primary button, outline secondary, focus ring-2 ring-zinc-950/10, use the stock shadcn/ui components

Neo-brutalism neo-brutalism: flat saturated fills (yellow #FFDC58, pink #FF90E8, cyan #A6FAFF), border-2 or border-[3px] solid black on everything, hard offset shadows (shadow-[4px_4px_0_0_#000]) with no blur that shrink on press, bold grotesque type like Archivo Black or Space Grotesk, rotated sticker badges

Editorial / magazine an editorial magazine look: off-white paper background (#F7F3EA), a high-contrast serif for headlines (Instrument Serif, Playfair Display or GT Sectra) with a clean sans for body, a centered masthead between double rules, a 12-column grid with 1px black rules between stories, small-caps section kickers in one red accent (#C8102E)

Notion-clean a Notion-clean look: white page with warm grays (text #37352F, muted #787774, sidebar #F7F7F5), Inter or the system UI font at 16px, a full-width cover image and large emoji page icon above a big bold title, property rows in muted gray, simple checkbox lists, gray callout blocks with an emoji, ⋮⋮ drag handles shown only on hover

Aurora gradients and glass aurora gradients with glass: dark background (#0B0B1A) with 3 large blurred color blobs (violet #7C3AED, cyan #06B6D4, pink #EC4899) slowly drifting, content on frosted panels (bg-white/10, backdrop-blur-xl, border border-white/20, rounded-2xl) with white text; ensure text contrast stays above 4.5:1

Y2K / retro web a Y2K retro web look: sky-blue to silver gradient background with ✦ sparkles, chrome gradient display text (Arial Black with a metallic linear-gradient via bg-clip-text and a dark outline), glossy pill buttons with a white highlight and outset bevel, a scrolling marquee strip, pixel-perfect 1px borders, a black hit counter in green digits

Oversized and kinetic typography an oversized, kinetic type layout: a display headline in a variable grotesk at clamp(4rem, 14vw, 12rem) with tight -0.04em tracking that bleeds off the right edge, a second line as outlined text (-webkit-text-stroke with transparent fill), a slowly scrolling marquee band of repeated words, letters that shift weight on scroll via font-variation-settings; small 14 to 16px body copy for contrast, motion off under prefers-reduced-motion

Terminal / dev-tool aesthetic a terminal / dev-tool aesthetic for the web: near-black background (#0B0E14), a monospace font for everything (JetBrains Mono, Geist Mono or Berkeley Mono), one phosphor accent (green #7EE787 or amber), a hero that types out a command with a blinking block cursor, a copyable install command in a $ prompt box, links styled as [docs] [github], features in 1px bordered boxes with box-drawing headers like ┌─ deploy ─┐, and keyboard shortcuts shown inline

Claymorphism claymorphism: pastel fills (lavender, mint, peach), very large radii (24 to 32px), each surface with three shadows (a soft outer drop shadow like 12px 12px 24px rgba(0,0,0,.12), an inset light highlight top-left and an inset darker shade bottom-right), chunky rounded type like Nunito or Baloo 2, puffy buttons that squash slightly on press, 3D clay-style illustrations; keep text contrast at WCAG AA despite the pastels

iOS

Tab bar a bottom tab bar (SwiftUI TabView with Tab items) with 4 top-level sections, SF Symbol icons with short labels and badges, each tab keeping its own NavigationStack, plus a search tab (Tab(role: .search))

Large title navigation bar a large-title navigation bar (NavigationStack with .navigationTitle and .navigationBarTitleDisplayMode(.large)) that collapses to a small inline title when the list scrolls, toolbar buttons at the trailing edge

Push navigation drill-down push navigation with NavigationStack and NavigationLink: list rows with trailing chevrons push a detail screen, a back button labeled with the previous title, and edge swipe to go back

Sheet with detents a sheet presented with .sheet and .presentationDetents([.medium, .large]), a visible grabber (.presentationDragIndicator(.visible)), Cancel and Add buttons in its toolbar; Liquid Glass sheet inset from the edges at the medium detent, the page behind receding only at the large detent

Full-screen cover present the flow with .fullScreenCover so it covers the whole screen including the status bar area, with an explicit close (xmark) or End button and no swipe-to-dismiss

Sidebar split view an iPad layout with NavigationSplitView: a sidebar of sections, a content list and a detail pane, the sidebar collapsible, falling back to a stack on iPhone

Search bar search with .searchable(text:), placed at the bottom as in iOS 26 (in the bottom toolbar, or a Tab(role: .search) in the tab bar), scope buttons via .searchScopes for categories, and live .searchSuggestions with the typed text highlighted, results updating as you type

Tab bar minimize and bottom accessory a TabView whose floating glass tab bar collapses to a single button on scroll (.tabBarMinimizeBehavior(.onScrollDown)), with a Now Playing mini player in .tabViewBottomAccessory that sits above the bar and moves inline when it minimizes

Search tab a dedicated search tab (Tab(role: .search)) that iOS sets apart at the trailing end of the tab bar; selecting it shows a bottom search field with .searchable on the TabView, suggestions and recent searches above it

iPadOS windowing and the menu bar iPadOS 26 windowing: multiple scenes (WindowGroup, UIApplicationSupportsMultipleScenes) with layouts that adapt to any window size, and a full menu bar defined with .commands (CommandMenu, CommandGroup) giving every action a keyboard shortcut

Popover on iPad a .popover(isPresented:attachmentAnchor:arrowEdge: anchored to its toolbar button on iPad, adapting to a sheet in compact width on iPhone (tune with .presentationCompactAdaptation)

Inset grouped list an inset grouped List (.listStyle(.insetGrouped)) or Form: rounded sections on the grouped background, rows with white SF Symbols on colored rounded-square icons, trailing values, chevrons and toggles, plus section headers and footers

Swipe actions list rows with .swipeActions: trailing red Delete (role: .destructive) and orange Flag, leading blue Mark as Read, full swipe performs the first action

Context menu a long-press .contextMenu with a preview (contextMenu(menuItems:preview:)): Copy, Share and Favorite, then a red destructive Delete in its own section

Confirmation dialog a .confirmationDialog with a short title, the destructive choice in red (role: .destructive) and a separate Cancel button

Segmented control a segmented control (Picker with .pickerStyle(.segmented)) of 2 to 5 short options that switches the view of the same data below it

Pickers native pickers: Picker with .pickerStyle(.menu) for a few options, DatePicker with .datePickerStyle(.graphical) for dates and .wheel for times

Alert an .alert with a short question as the title, a one-line explanation, a Cancel button (role: .cancel) and a red destructive action (role: .destructive)

SF Symbols SF Symbols for every icon (Image(systemName:)), matched to the adjacent text weight, with .symbolRenderingMode(.hierarchical) and .symbolEffect animations where they help

Pull to refresh pull to refresh on the list with .refreshable { await reload() }, showing the native spinner under the large title

Pull-down menu a pull-down Menu from a toolbar button (SwiftUI Menu, or UIButton with .menu and showsMenuAsPrimaryAction): grouped Sections with SF Symbols, a Sort By submenu with a Picker for the checkmarked choice, and a destructive item last

Share sheet a share button using ShareLink(item:subject:message:preview:) (UIActivityViewController in UIKit) with the square.and.arrow.up symbol, sharing a URL or Transferable item with a title and thumbnail preview

Page control and paged carousel a horizontally paged carousel (TabView with .tabViewStyle(.page) and .indexViewStyle(.page(backgroundDisplayMode: .always)), or a ScrollView with .scrollTargetBehavior(.viewAligned) and peeking neighbors) with page dots showing the current page

TipKit tips feature tips with TipKit: a Tip with a title, message and SF Symbol image, shown as a popover on the button it explains (.popoverTip(tip)) or inline (TipView(tip)), Tips.configure() at launch, with rules and invalidation so each tip shows once

Grouped toolbar items iOS 26 toolbars with related buttons grouped in shared glass capsules (ToolbarItemGroup), ToolbarSpacer(.fixed) or ToolbarSpacer(.flexible) between groups, and the Done action as a tinted checkmark using .buttonStyle(.glassProminent) (or Button(role: .confirm)) at the trailing edge

Home Screen widgets a WidgetKit widget in small and medium sizes (.systemSmall, .systemMedium) showing one glanceable value, refreshed by a TimelineProvider, tapping deep-links into the app

Live Activities and the Dynamic Island a Live Activity (ActivityKit) with a Lock Screen card showing ETA and a progress track, plus Dynamic Island compact leading/trailing, minimal and expanded layouts

Notifications local and push notifications (UserNotifications) with clear titles, thread grouping (threadIdentifier), actionable buttons, and the right interruption level (.passive, .active, .timeSensitive)

App icon an app icon built as layers in Icon Composer: one simple bold glyph on a gradient background, with default, dark, clear and tinted appearances, no text, no baked-in rounded corners

Haptics subtle haptics with .sensoryFeedback: .selection on picker changes, .success after saving, .error on failure, .impact when something snaps into place; never on every tap

Controls a ControlWidget (WidgetKit, iOS 18 and later) with a ControlWidgetToggle or ControlWidgetButton backed by an App Intent, usable in Control Center, on the Lock Screen and with the Action button, with an SF Symbol, a short title and a tint for the on state

Liquid Glass the iOS 26 Liquid Glass look: a floating glass tab bar that minimizes on scroll (.tabBarMinimizeBehavior(.onScrollDown)), toolbar buttons grouped in glass capsules, custom floating controls with .glassEffect() inside a GlassEffectContainer, content running edge to edge beneath

Blur materials and vibrancy translucent backgrounds using SwiftUI Material (.ultraThinMaterial through .thickMaterial) with hierarchical foreground styles (.primary, .secondary) instead of fixed colors, so text stays readable over photos

Dynamic Type support Dynamic Type everywhere: semantic text styles (.font(.body), .headline) rather than fixed sizes, @ScaledMetric for icon sizes and spacing, and layouts that stack vertically at accessibility sizes (ViewThatFits or dynamicTypeSize.isAccessibilitySize)

SF Pro and the type scale the system font with semantic text styles only (.largeTitle, .title, .headline, .body, .subheadline, .footnote, .caption), bold large titles, no hard-coded font sizes

Tint and semantic colors a single app accent color set as the AccentColor asset and .tint, used only for interactive elements, with semantic system colors (Color(.label), .secondaryLabel, .systemBackground, .separator) everywhere else so dark mode works

Spring animations and zoom transitions spring animations for all motion (.spring(duration: 0.4, bounce: 0.2), withAnimation(.snappy)), and zoom transitions from thumbnails to detail screens (.navigationTransition(.zoom(sourceID:in:)) with .matchedTransitionSource)

Scroll edge effect let content scroll under floating glass bars using the iOS 26 scroll edge effect: .scrollEdgeEffectStyle(.soft, for: .top) by default, .hard where pinned headers or dense text need a crisper edge, and custom bars added with .safeAreaBar(edge:) so they get the effect

Concentric corners concentric corners for nested shapes: ConcentricRectangle() or .rect(corners: .concentric) for cards and buttons inside sheets and containers, with .containerShape() setting the outer shape, instead of hard-coded corner radii

Android

Top app bar a Material 3 top app bar: LargeTopAppBar on top-level screens that collapses to a 64dp small bar on scroll (TopAppBarDefaults.exitUntilCollapsedScrollBehavior), CenterAlignedTopAppBar or TopAppBar on detail screens, navigation icon on the left and at most 2 to 3 action icons on the right

Navigation bar a Material 3 navigation bar (Compose NavigationBar with NavigationBarItem) with 4 destinations, icon plus label on each, a tonal pill active indicator behind the selected icon, filled icon when selected and outlined otherwise

Navigation rail a Material 3 navigation rail (Compose NavigationRail, or NavigationSuiteScaffold to switch automatically between bar and rail by window size class) on medium and expanded widths: 80dp wide, menu icon and FAB at the top, destinations with pill indicators below

Navigation drawer a Material 3 modal navigation drawer (Compose ModalNavigationDrawer with ModalDrawerSheet and NavigationDrawerItem) opened from the top app bar menu icon or an edge swipe, scrim over the content, full-width rounded active indicator on the current item, section headers and item badges

Predictive back support predictive back (android:enableOnBackInvokedCallback="true"): use Navigation Compose so screen transitions follow the gesture, and PredictiveBackHandler for custom surfaces like sheets and search, scaling and shifting the outgoing screen with the swipe progress

Edge-to-edge edge-to-edge layout: call enableEdgeToEdge(), let the hero image and app bar background draw behind the status bar, apply WindowInsets.safeDrawing padding (Scaffold contentWindowInsets) to text and buttons, and let lists scroll under a transparent gesture navigation bar

List-detail layout an adaptive list-detail layout (Compose ListDetailPaneScaffold from material3-adaptive): single pane on compact widths, list and detail side by side at 600dp and up, the selected list item highlighted, navigation rail on the left, back from detail returns to list on phones

Floating and docked toolbars a Material 3 Expressive floating toolbar: HorizontalFloatingToolbar centered near the bottom (offset FloatingToolbarDefaults.ScreenOffset) with 4 to 5 icon toggles and a FAB attached via the floatingActionButton slot, hiding on scroll down (FloatingToolbarDefaults.exitAlwaysScrollBehavior); use a docked toolbar (FlexibleBottomAppBar) instead of BottomAppBar when the actions should span the full width

Window size classes and the supporting pane an adaptive layout driven by window size classes (currentWindowAdaptiveInfo().windowSizeClass, breakpoints 600 / 840 / 1200 / 1600dp): NavigationSuiteScaffold to switch from navigation bar to rail automatically, ListDetailPaneScaffold for list and detail at expanded width, and SupportingPaneScaffold to show related content in a side pane on wide windows that moves below or into a sheet on compact ones

Foldable postures hinge-aware layouts using Jetpack WindowManager (WindowInfoTracker.windowLayoutInfo): when a FoldingFeature is HALF_OPENED and HORIZONTAL (tabletop), put the video or viewfinder above the fold and controls below it; when VERTICAL (book), split into two panes at the fold bounds; keep touch targets and text off the hinge when isSeparating is true

Floating action button a Material 3 floating action button bottom-right with 16dp margins for the primary action: ExtendedFloatingActionButton with icon and label that collapses to a 56dp FloatingActionButton on scroll (expanded = !listState.isScrollInProgress), primaryContainer color, 16dp corner radius

Bottom sheet a Material 3 modal bottom sheet (Compose ModalBottomSheet with rememberModalBottomSheetState) with a drag handle, 28dp top corners, surfaceContainerLow color and a scrim; swipe down or tap the scrim to dismiss, and it supports predictive back

Snackbar a Material 3 snackbar via Scaffold snackbarHost and SnackbarHostState.showSnackbar(message, actionLabel = "Undo"), inverse surface colors, one action at most, shown above the navigation bar and pushing the FAB up

Chips Material 3 chips: a horizontally scrolling row of FilterChip with a leading checkmark when selected, AssistChip with a leading icon for contextual actions, InputChip with avatar and trailing close for entered values, SuggestionChip for quick replies; 32dp tall, 8dp corners

Cards Material 3 cards with 12dp corners: Card (filled, surfaceContainerHighest) for grouped content, ElevatedCard for items that need to lift off a busy background, OutlinedCard in dense layouts; the whole card is one tap target with a ripple

Dialog a Material 3 basic dialog (Compose AlertDialog) with 28dp corners and surfaceContainerHigh color: optional centered icon, a question as the headline, one sentence of supporting text, and text buttons bottom-right with the confirming verb ("Discard") on the right and "Cancel" on its left

Segmented buttons (legacy) Material 3 segmented buttons: SingleChoiceSegmentedButtonRow with SegmentedButton for a Day / Week / Month switcher, MultiChoiceSegmentedButtonRow for multi-select filters, 40dp tall, checkmark icon on selected segments

Search bar and search view a Material 3 search bar (Compose SearchBar) at the top: 56dp pill on surfaceContainerHigh with a leading icon and the account avatar trailing; on tap it expands to a full-screen search view with back arrow, recent searches with history icons and suggestions; use DockedSearchBar on tablets

Ripple and state layers Material 3 interaction states on every clickable element: a bounded ripple from the touch point (Modifier.clickable with ripple()) and state layers of the content color at 8% hover, 10% focus, 10% pressed and 16% dragged, plus a visible focus ring for keyboard users

Button groups Material 3 Expressive button groups: a standard ButtonGroup of icon ToggleButtons that widen and morph from round to square when pressed (animateWidth), and a connected button group for single-select view switching, built from ToggleButtons with ButtonGroupDefaults.connectedLeadingButtonShapes(), connectedMiddleButtonShapes() and connectedTrailingButtonShapes(), 2dp apart, the checked button fully rounded with a check icon

Split button a Material 3 Expressive split button (Compose SplitButtonLayout with SplitButtonDefaults.LeadingButton and SplitButtonDefaults.TrailingButton): the leading half runs the default action, the trailing chevron half toggles a DropdownMenu of variants, the trailing half morphs to a circle and the chevron rotates 180 degrees while open; filled, tonal, outlined and elevated variants

FAB menu a Material 3 Expressive FAB menu (Compose FloatingActionButtonMenu with a ToggleFloatingActionButton and FloatingActionButtonMenuItem entries): 2 to 6 labeled pill items stacked above the FAB in primaryContainer color, the FAB morphing from rounded square to a circle with a close icon when expanded, predictive back and tapping outside collapse it

Carousel a Material 3 carousel: HorizontalMultiBrowseCarousel (rememberCarouselState, preferredItemWidth 186dp) with items clipped by Modifier.maskClip(MaterialTheme.shapes.extraLarge) so large, medium and small items reshape as they scroll; HorizontalCenteredHeroCarousel for one featured item and HorizontalUncontainedCarousel for equal-width items; text over images only on the large item

Loading indicator the Material 3 Expressive LoadingIndicator for short indeterminate waits (shape morphing through MaterialShapes), ContainedLoadingIndicator on a primaryContainer circle as the pull-to-refresh indicator (PullToRefreshBox with indicator = PullToRefreshDefaults.LoadingIndicator), and wavy LinearWavyProgressIndicator when progress is known

Dynamic color Material You dynamic color: on Android 12+ use dynamicLightColorScheme(context) / dynamicDarkColorScheme(context) in MaterialTheme, with a fallback color scheme generated from the brand color in Material Theme Builder; style everything with color roles (primary, primaryContainer, surfaceContainer, onSurfaceVariant) instead of hex values

Material 3 Expressive the Material 3 Expressive style (MaterialExpressiveTheme with MotionScheme.expressive()): emphasized bold headlines, MaterialShapes such as Cookie9Sided and Clover4Leaf for avatars and accents, ButtonGroup buttons that morph from pill to square when pressed, LoadingIndicator and wavy progress indicators, spring motion with a slight overshoot

Shape scale use the Material 3 shape scale only (MaterialTheme.shapes): extraSmall 4dp for menus and snackbars, small 8dp for chips, medium 12dp for cards, large 16dp for FABs and navigation drawers, extraLarge 28dp for dialogs and bottom sheets, full (CircleShape) for buttons and search bars

Tonal elevation tonal elevation instead of drop shadows: use the surface container roles (surface, surfaceContainerLow, surfaceContainer, surfaceContainerHigh, surfaceContainerHighest) for stacked layers, and let the top app bar switch to surfaceContainer when content scrolls under it (TopAppBarDefaults scrolledContainerColor)

Type scale text styled only with Material 3 type roles (MaterialTheme.typography): displayLarge for hero numbers, headlineMedium for screen headings, titleLarge for app bar and section titles, bodyLarge for reading text, labelLarge for buttons; sizes in sp so they follow the system font scale

Material Symbols icons from Material Symbols Rounded, imported as vector drawables and shown with Icon(painterResource(...)), 24dp, weight 400; outlined (FILL 0) for unselected and filled (FILL 1) for selected navigation items

Widgets and themed icons a home screen widget built with Jetpack Glance (GlanceAppWidget with GlanceTheme for dynamic colors): rounded corners matching system_app_widget_background_radius, resizable layouts for small and large sizes, tapping opens the matching screen; plus a monochrome layer in the adaptive launcher icon for themed icons

Container transform a container transform from list card to detail screen using Compose SharedTransitionLayout with Modifier.sharedBounds on the card and the detail container, sharedElement on the image, corner radius animating from 12dp to 0, and the reverse on back

Watch & spatial

Smart Stack widgets a watchOS Smart Stack widget (WidgetKit, .accessoryRectangular) showing one glanceable value in the app's color, with relevance (RelevanceConfiguration in watchOS 26, or RelevantIntent) so it rises to the top of the stack when timely; tapping opens the app

Complications watch face complications built with WidgetKit accessory families (.accessoryCircular, .accessoryCorner, .accessoryRectangular, .accessoryInline), using Gauge and .widgetLabel, and rendering well in the tinted watch face mode (widgetRenderingMode .accented)

Vertical paging with the Digital Crown a vertically paged layout (TabView with .tabViewStyle(.verticalPage)) that the Digital Crown moves page by page, one full-screen view per page with a containerBackground color, and .digitalCrownRotation for any value the crown should adjust directly

Glanceable list and detail a watchOS NavigationStack with a List of large rows (title plus one detail line) that pushes a detail screen whose main action is a big full-width button (.buttonStyle(.borderedProminent)) that also responds to double tap (.handGestureShortcut(.primaryAction)), with secondary actions in the toolbar

Short look and long look notifications watchOS notifications: a short look with the app icon, title and app name, and a long look (default, or a custom SwiftUI interface via WKUserNotificationHostingController) with the full message, an optional image, and a few action buttons from the notification category

Edge-hugging button a Wear OS Material 3 EdgeButton for the main action, placed in the edgeButton slot of ScreenScaffold below a TransformingLazyColumn so it hugs the bottom bezel and works with the list scroll; EdgeButtonSize.Medium with a short label or a single icon, primary color

Curved, transforming list a Wear OS list using TransformingLazyColumn inside ScreenScaffold (shared rememberTransformingLazyColumnState for the curved ScrollIndicator), each item with Modifier.transformedHeight(this, transformationSpec) and transformation = SurfaceTransformation(transformationSpec), full-width pill Buttons, a ListHeader, curved TimeText at the top and rotary crown scrolling

Tiles a Wear OS tile built with TileService and ProtoLayout Material 3 primaryLayout: an icon and short title in titleSlot, one large glanceable value with a circular progress arc in mainSlot, and an edge button action (textEdgeButton or iconEdgeButton) in bottomSlot, using dynamic system colors and refreshing on a freshness interval

Wavy progress and the round screen round-screen progress for Wear OS: a full-screen CircularProgressIndicator along the bezel with a gap at the top for curved TimeText, or a smaller ring around the play/pause button for media, with the expressive wavy style while playing; use SegmentedCircularProgressIndicator for step goals, and keep the key number large in the center

Windows, volumes and immersive spaces a visionOS app that starts in a glass window (WindowGroup), opens a 3D model in a volume (.windowStyle(.volumetric) with a RealityView), and offers an ImmersiveSpace (.immersionStyle(.mixed, .progressive, .full)) opened with openImmersiveSpace for focused moments

Ornaments visionOS ornaments: a TabView that renders as the vertical tab bar ornament on the window's leading edge, and the main controls in a bottom ornament (.toolbar with ToolbarItem(placement: .bottomOrnament), or .ornament(attachmentAnchor: .scene(.bottom))) overlapping the window edge

Look-to-highlight and indirect pinch every interactive element uses system buttons or .hoverEffect() (with .contentShape for the hit area) so it highlights when looked at, targets at least 60 pt with spacing between them, actions on the standard tap gesture (indirect pinch), and no custom gaze tracking since eye position is private

Glass windows that adapt to the room visionOS windows on the system glass material (.glassBackgroundEffect()), white primary text with vibrant secondary styles (.secondary, .tertiary), no opaque or solid-color backgrounds, so the window picks up the light and color of the room around it

Desktop apps

Menu bar and app menus a complete native menu bar (File, Edit, View, Window, Help) with the standard items, shortcuts right-aligned, checkmarks on toggles and unavailable items grayed out (SwiftUI .commands / Electron Menu.setApplicationMenu)

Unified toolbar a unified title bar and toolbar in one row: window controls, sidebar toggle, title with a subtitle, then a few icon-only toolbar buttons and a search field on the right (SwiftUI .toolbar + .searchable, .windowToolbarStyle(.unified))

Three-pane layout a three-pane layout: a collapsible sidebar of sources with unread counts, a scrollable list of items showing title, preview and time, and a detail pane for the selected item, with dividers you can drag to resize (SwiftUI NavigationSplitView with sidebar, content and detail columns)

Inspector panel a collapsible inspector panel on the right showing the selected object's properties in labeled sections (Layout, Fill, Opacity) with compact numeric fields and sliders that update the canvas live (SwiftUI .inspector(isPresented:))

Menu bar extra a menu bar extra with no Dock icon: a monochrome template icon that opens a small panel with status at the top, a few recent items, a pause toggle, and Open and Quit at the bottom (SwiftUI MenuBarExtra with .menuBarExtraStyle(.window), LSUIElement = YES)

Keyboard launcher a keyboard launcher: a floating centered panel opened by a global hotkey, a large search field, results grouped by type with icons and the matched letters bold, the first result preselected, ↑ ↓ to move, Return to run, ⌘K for more actions on the selection, Esc to dismiss

Document tabs and windows a document-based app with native window tabs: each document opens in a tab, tabs reorder by dragging, a tab dragged out becomes its own window and can be merged back, ⌘T opens a new tab and ⌘W closes one (SwiftUI DocumentGroup / NSWindow.tabbingMode)

Context menu right-click context menus on every item with the relevant actions grouped by separators, shortcuts right-aligned, submenus marked with ›, and destructive actions last in red (SwiftUI .contextMenu)

Settings window a native Settings window opened with ⌘, : icon tabs in the toolbar (General, Accounts, Appearance, Advanced), a centered form with right-aligned labels, and every change applied instantly without a Save button (SwiftUI Settings scene with a TabView of Forms)

Status bar a thin status bar along the bottom of the window: workspace state on the left (branch, sync, error and warning counts), current-file details on the right (line and column, encoding, language), each item clickable with a tooltip

Drag and drop drag and drop for moving items: a semi-transparent drag preview with a count badge, a highlighted drop target, an insertion line when reordering, a green + badge when Option copies instead of moves, and files from other apps accepted (SwiftUI .draggable / .dropDestination)

Windows custom title bar a Windows 11 custom title bar with the WinUI 3 TitleBar control (Windows App SDK 1.7): ExtendsContentIntoTitleBar = true, SetTitleBar(titleBar), back button, app icon, title and subtitle, an AutoSuggestBox in the Content slot for search, the account PersonPicture in the RightHeader, standard caption buttons, the empty areas left as drag regions, and Window.SystemBackdrop = new MicaBackdrop()

Windows NavigationView, InfoBar and TeachingTip a WinUI 3 app shell: NavigationView with PaneDisplayMode="Auto" (Left when wide, LeftCompact or LeftMinimal when narrow), NavigationViewItems with icons and an InfoBadge count, IsSettingsVisible for the built-in Settings item; an InfoBar at the top of the page (Severity="Warning", Title, Message, an ActionButton, IsClosable) for app status; and a TeachingTip with Target set to a new toolbar button, Title, Subtitle and a close button, shown once

GNOME header bar and adaptive split view a GTK 4 / libadwaita 1.7 layout: AdwNavigationSplitView with an AdwToolbarView and AdwHeaderBar in each pane, the sidebar as a navigation-sidebar list, an AdwBreakpoint (max-width: 400sp) setting collapsed = true so it becomes a single page with a back button, an AdwToggleGroup in the content header bar to switch Edit and Preview, and AdwToastOverlay toasts with an Undo button

macOS inset glass sidebar and sheets a macOS 26 (Tahoe) window with SwiftUI NavigationSplitView: the system inset Liquid Glass sidebar with traffic lights inside it, the detail's hero image using .backgroundExtensionEffect() so it extends mirrored and blurred under the sidebar, toolbar items in glass capsules over the content, and window-attached sheets via .sheet with capsule Cancel and default buttons; no custom sidebar background

Native Mac app a native Mac app feel: SwiftUI or AppKit with standard controls only, a translucent sidebar (NavigationSplitView), unified toolbar, SF Pro at system sizes, the system accent color, a full menu bar with shortcuts, light and dark mode, Liquid Glass on macOS 26, and no web-style custom chrome

Windows 11 / WinUI a native Windows 11 app: WinUI 3 (Windows App SDK) with a Mica backdrop (MicaBackdrop), NavigationView on the left, settings grouped in rounded cards (SettingsCard from the Windows Community Toolkit), Segoe UI Variable, the system accent color, Acrylic on flyouts, and an extended title bar with standard caption buttons

GNOME / libadwaita a GNOME app built with GTK 4 and libadwaita: AdwHeaderBar with flat buttons and a centered title, AdwPreferencesGroup boxed lists of AdwActionRow and AdwSwitchRow, a pill-shaped suggested-action button, AdwToast for confirmations with Undo, following the GNOME HIG

Pro app dark UI a pro-app dark UI: neutral dark grays (not pure black), dense dockable panels with 11–12px labels, scrubbable numeric fields, one accent color for selection and a red playhead, the content viewer as the brightest element, and a keyboard shortcut for every action

AI interfaces

Streaming response with stop button stream the reply token by token and render Markdown as it arrives, with a soft caret at the end; while streaming, the send button turns into a square Stop button (Esc also stops) and a stopped reply keeps its partial text; auto-scroll only while the user is at the bottom, otherwise show a round "jump to latest" arrow

Prompt starters and follow-up chips an empty state with a short greeting and 3 to 4 prompt starter cards specific to this product (a tap fills and sends the prompt), plus 2 to 3 follow-up suggestion chips under each finished answer, generated from the answer and hidden while streaming

Context chips and attachments a composer with an attachments row of removable chips (file icon, name, ✕; images as thumbnails) above the text, a paperclip button and drag-and-drop to add files, and an @-mention popover that filters files, docs and connected sources as you type, inserting the choice as an inline chip

Model and mode picker a model picker as a compact dropdown in the composer's bottom-right showing the current model name, opening a menu with each model's one-line strength and a check on the active one; mode toggles (Think, Research, Plan) as pill buttons beside the + menu that light up when on and persist per chat

Regenerate, edit and branch edit-and-branch history: a pencil on each user message opens it inline with Cancel / Save, retry on each reply, and every edit or retry creates a sibling branch shown with a "‹ 2 / 3 ›" pager under the message; switching branches swaps everything below it

Voice mode a full-screen voice mode: a centered animated orb that pulses with the user's mic level and ripples while the model speaks, a live two-speaker transcript below with the current words highlighted, barge-in to interrupt, and large round Mute, captions and red End buttons at the bottom

Reasoning disclosure a reasoning disclosure above each answer: a shimmering "Thinking…" label with elapsed time while it reasons, which collapses to "Thought for 12s ›" when the answer starts; expanding shows the reasoning summary in smaller muted text with a left rule, never mixed into the answer

Agent trace an agent trace: a vertical timeline of tool calls, each row an icon, a short human summary ("Read checkout.ts", "Ran npm test · 2 failed"), duration and a status mark (spinner, ✓, ×); rows expand to show the exact input and output in monospace; the current step shimmers, and a header shows step count, elapsed time and a Stop button

Plan with live checklist a live plan checklist the agent writes first and updates as it goes: ☒ done items struck through and dimmed, the in-progress item bold with a spinner, ☐ pending items plain, a "3 of 5" count; in plan mode the agent stops after writing the plan and asks for approval before editing anything

Approval gate an approval gate before side-effecting tool calls: an inline card naming the tool and the exact action (recipient, command or amount in monospace), with Deny, Allow once and Always allow for this tool; the agent waits without timing out, Deny asks "what should I do instead?", and "always" rules are listed and revocable in settings

Checkpoints and restore automatic checkpoints before each agent turn: a "Restore checkpoint" button on hover between messages, a confirm popover listing the files that will revert ("Restore 3 files and drop 2 messages?"), options to restore code only, chat only or both, and a toast with Redo after restoring

Background agent task background agent tasks: a task list with a status for each (Queued, Running with a progress bar and current step, Needs input, Ready to review), safe to close the app, plus a push notification and an in-app badge when a task finishes or needs a decision; opening a finished task shows its summary and diff to review

Inline citations and sources inline citations: small numbered pills right after each supported sentence, a hover card previewing the source (favicon, site, title, quoted snippet), and a Sources panel listing every source in order with favicon and domain; uncited claims stay visually plain, never get a fake marker

Message actions and feedback message actions under each finished reply: icon buttons for copy (with a "Copied" confirmation), thumbs up, thumbs down, retry and more, hidden while streaming; thumbs down opens a small sheet with reason chips (Inaccurate, Didn't follow instructions, Too long, Unsafe) and an optional comment, logged with the message id

Usage and context meters usage and context meters: a small ring in the composer showing context used (tooltip "124k of 200k tokens"), a usage popover with bars for the current session and the week with reset times, and an amber banner above the composer at 80% ("You're close to your limit, resets at 3:00 PM") that offers the next step instead of only blocking

Memory and incognito chat visible memory: an inline "Memory updated" chip when something is saved (tap to view or forget it), a Memory settings page listing every saved fact with delete, and an incognito toggle in the chat header that switches to a visibly different dark, ghost-icon chat that is not saved to history or memory

AI disclosure and confidence cues AI disclosure: an "AI summary" label with a sparkle on every generated block, a muted one-line caveat ("AI can make mistakes. Check important info."), low-confidence or unsourced phrases with an amber dotted underline and a tooltip saying why, and Use / Edit / Dismiss actions so a person decides

Artifact / canvas side panel an artifact side panel: when the reply produces a document, app or diagram, open it in a resizable right panel (chat stays on the left, about 40%) with a title, version switcher, Preview / Code tabs, Copy and Publish buttons; follow-up messages update the artifact in place as a new version, and the chat shows a small card that reopens it

Generative UI components generative UI in the chat: the model calls tools whose results render as real components (controlled: a fixed set of typed components like WeatherCard or FlightOptions with props from the tool result; declarative: a JSON spec drawn by our renderer with our design tokens; open-ended: sandboxed HTML in an iframe only for one-off visuals), each with a loading skeleton and a text fallback

Inline AI actions on selection inline AI on selection: selecting text shows a floating toolbar with an "Ask AI" input and quick actions (Improve, Shorten, Fix grammar, Translate); Cmd-K opens the same input at the cursor; the result streams in place as a highlighted suggestion with Accept, Discard and Try again, and Esc cancels

Ghost-text autocomplete ghost-text suggestions: after a short typing pause, show the predicted continuation in faded gray italic inline after the cursor (multi-line allowed), Tab to accept all, Cmd-→ to accept word by word, Esc or keep typing to dismiss, with a small "Tab to accept" hint the first few times

Diff review of AI edits AI edits as a reviewable diff: red removed and green added lines inline in the editor, Accept / Reject buttons on each hunk (with shortcuts), a bar at the top with "3 files changed, 2 of 5 hunks reviewed", Accept all / Reject all, and arrows to jump between hunks; nothing is written until accepted

Screens & states

Onboarding a short onboarding flow: at most 3 screens with page dots, a Skip link on every screen, one personalization question with selectable chips, and a single primary Continue button; ask for sign-in and permissions only when first needed, not up front

Sign-in a sign-in screen that leads with passkeys (WebAuthn / ASAuthorizationController / Credential Manager), then Sign in with Apple and Continue with Google buttons in their official styles, then an email field that sends a magic link or 6-digit code; no password field, autofill hints set on every input

Permission priming permission priming: ask in context, when the feature is first used, with an in-app screen that shows the benefit in one sentence, a primary "Turn on" button that triggers the system prompt and a "Not now" that skips it; if denied, show a later inline hint that deep-links to Settings

Settings screen a settings screen: options grouped under short section headers, one row per option with label, optional helper text and a right-aligned control (toggle, current value, or chevron); toggles save instantly with no Save button; inset grouped List on iOS, preference rows on Android, left section sidebar on desktop web

Search screen a dedicated search screen: focused field with Cancel, recent searches (each removable with ✕, plus Clear all) and suggestion chips before typing, live debounced results while typing, and a "No results for …" state with spelling suggestions (SwiftUI .searchable with .searchSuggestions, Material 3 SearchBar on Android, combobox ARIA pattern on web)

Checkout a multi-step checkout: a stepper showing completed, current and upcoming steps, one topic per step, a sticky order summary beside the form on desktop (collapsible at the top on mobile), Back without losing input, inline validation on blur, and Apple Pay / Google Pay express buttons above the form

Paywall a paywall: close ✕ always visible, 3 to 4 benefit lines with checkmarks, yearly and monthly plan cards with yearly preselected and a "Save 45%" badge, a primary trial button, the exact price and renewal terms under it, and Restore Purchases / Terms / Privacy links (StoreKit 2 SubscriptionStoreView on iOS, Google Play Billing on Android)

Feed a feed of post cards (avatar, name, timestamp, text, optional media, action row), infinite scroll with a skeleton for the next page, pull-to-refresh on mobile, and a floating "↑ 4 new posts" pill instead of inserting new items above the reading position (LazyColumn / List / virtualized list)

Detail page a detail page: large media gallery with thumbnails, title, price and rating up top, option pickers (color swatches, size chips), one prominent primary action that stays visible (sticky bottom bar on mobile, right column on desktop), and long content (details, shipping, reviews) in collapsible sections

Chat / messaging a chat thread: incoming bubbles left in gray, outgoing right in the accent color, consecutive messages grouped with tight spacing, centered date separators, sent/delivered/read status on the last outgoing message, a typing indicator, and a composer pinned above the keyboard (inverted list anchored to the bottom, imePadding on Android)

AI chat interface v2.14

Error summary a GOV.UK-style error summary: on submit with errors, show a box at the top of the form with a thick red border, the heading "There is a problem", and a list of links (one per error, same wording as the field message) that move focus to the field; move focus to the summary on load (tabindex="-1", role="alert"), prefix the page <title> with "Error: ", and mark each invalid field with a red left bar, a red border and an "Error: Enter your date of birth" message above the input (aria-describedby)

Check your answers a check-your-answers page: heading "Check your answers before sending your application", answers grouped under section headings in a summary list (key in bold, value, right-aligned "Change" link with visually hidden text like "Change date of birth"), Change returns to that question and back to this page after, then a "Now send your application" section with a short declaration and a primary "Accept and send" button

Task list page a task list hub page: title, "You have completed 2 of 5 sections" summary, tasks grouped under numbered headings, each row a link to that section with a status tag on the right (Completed in plain text, In progress and Not yet started as colored tags, Cannot start yet in gray and not linked), progress saved so people can leave and return, and a final "Check and send" task that unlocks when the rest are complete

Account setup an account setup flow after sign-up: 2 to 3 short steps with a segmented progress bar and Skip on optional steps; step 1 profile with avatar upload (initials placeholder, camera badge), display name prefilled from the sign-in provider, and a username field that checks availability as you type (✓ available / taken with suggestions); step 2 a single question with selectable cards ("What will you use it for?") that tailors the first screen; Continue disabled until required fields are valid; everything editable later in settings

Subscription management and cancel flow a subscription management page and an honest cancel flow: billing settings show the plan, price, next renewal date, payment method and invoices, with "Cancel subscription" as a visible button; one confirmation step says what happens and when ("You keep Pro until Apr 3, then move to Free; nothing is deleted"), optionally offers a pause or downgrade once, asks for a reason as optional, and gives "Keep Pro" and "Cancel subscription" equal visual weight; no confirmshaming, no hidden links, no forced phone call, and a confirmation email afterwards

Empty state designed empty states for every list and view: a small neutral illustration, a one-line title saying what goes here, one sentence of help, and the primary action that fills it (plus a secondary like Import); distinguish first-use empty, no-search-results and all-caught-up

Skeleton loading skeleton screens while loading: gray placeholder blocks matching the real layout's sizes (so nothing shifts), a slow pulse or shimmer, real headers and navigation shown immediately, skeletons only after a ~300 ms delay to avoid flashing (SwiftUI .redacted(reason: .placeholder), shadcn Skeleton)

Optimistic UI optimistic updates: apply the change to the UI instantly, mark pending items subtly (lower opacity or a tiny clock), send the request in the background, and on failure roll back with a toast or inline "Couldn't save · Retry" (React useOptimistic or TanStack Query onMutate/onError rollback)

Error state with retry error states scoped to the region that failed (never a blank page): an icon, a plain-language title like "Couldn't load invoices", one sentence on why and whether data is safe, a Try again button that retries in place, a link to status or support, and a copyable error ID; inline field errors stay next to their field

Offline state offline handling: a non-blocking banner ("You're offline. Changes will sync when you reconnect."), cached content stays readable, edits queue locally and show a pending marker, online-only actions are disabled with a tooltip, and the banner turns into "Back online" then hides (NWPathMonitor on iOS, ConnectivityManager on Android, navigator.onLine plus a service worker on web)

Undo instead of confirm undo instead of confirmation dialogs: perform archive/delete immediately, show a snackbar or toast "Conversation archived · Undo" for 5–8 seconds, soft-delete on the server until it expires, and support ⌘Z / Ctrl+Z on desktop; keep a typed confirmation only for irreversible actions

Badges and dots badges for unread items: a red count badge on tab and app icons capped at "99+", a plain dot when the count doesn't matter, a blue unread dot on list rows, badges cleared as soon as the item is seen (SwiftUI .badge(), Material 3 BadgedBox, UNNotificationContent.badge); never badge for marketing

Inline editing inline editing: hovering an editable value shows a subtle outline and pencil, click or Enter turns it into a field with the text selected, Enter or blur saves (optimistically), Escape cancels, invalid input shows an inline message without leaving edit mode; keyboard reachable with a visible focus ring

Progressive disclosure progressive disclosure: show only the 3–5 essential fields by default with sensible defaults, put the rest under a collapsed "Advanced options" disclosure (<details>/<summary> on web, DisclosureGroup in SwiftUI) that remembers its state, and never hide anything required to finish the task

Practices & systems

Follow platform conventions follow each platform's conventions instead of one custom look: on iOS a large-title nav bar, ‹ back with swipe-back, bottom tab bar and SF Symbols; on Android a top app bar with ← back, Material 3 navigation bar and FAB, and predictive back; on the web real links, browser back and standard keyboard shortcuts; use native controls (pickers, switches, sheets) wherever one exists

Touch targets touch targets of at least 44×44 pt on iOS and 48×48 dp on Android (44×44 CSS px on touch web, never under WCAG 2.2's 24×24 px), even for small icons, by padding the hit area rather than growing the icon; at least 8 pt/dp between adjacent targets

Color contrast color contrast meeting WCAG 2.2 AA in both themes: at least 4.5:1 for body text and placeholders, 3:1 for text 24px+ (or 18.66px bold) and for icons, borders and focus rings; check muted grays and white-on-accent buttons with a contrast checker, and never use color alone to carry meaning

Dark mode with semantic tokens light and dark themes from semantic color tokens (surface, surface-raised, text, text-muted, border, accent) with a value for each mode; follow the system setting (prefers-color-scheme / UITraitCollection / isSystemInDarkTheme) with a manual override; dark surfaces are dark gray (about #121212 to #1C1C1E, not pure black), elevation shown by lighter surfaces instead of shadows, accents desaturated slightly

Responsive breakpoints mobile-first responsive layout: single column below 640px, two columns from 768px, sidebar plus three-column content from 1024px, max content width around 1200px; navigation moves from a bottom bar or menu to a sidebar; use container queries for components, and Material window size classes (compact / medium / expanded) or iOS size classes in native apps

Design tokens use design tokens in three layers: primitives (blue-600 = #2563EB, space-4 = 16px, radius-md = 8px), semantic aliases (color.action.primary, color.text.muted, space.inset.md) and component tokens (button.bg); components reference only semantic tokens, never raw hex or px values; define them as CSS variables / a Tailwind theme / an asset catalog

8-point spacing grid an 8-point spacing system: every margin, padding and gap from the scale 4, 8, 12, 16, 24, 32, 48, 64; 16 px screen edge padding on mobile, 24+ on desktop; related items closer together (8) than separate groups (24); component heights on the grid too (32/40/48)

Type scale a type scale of 5–7 named styles used everywhere, e.g. Large Title 34, Title 22, Headline 17 semibold, Body 17, Callout 16, Footnote 13, Caption 12 (iOS Dynamic Type) or Material 3 display/headline/title/body/label; body at 16–17px with 1.4–1.5 line height, one font family, hierarchy from size and weight; support Dynamic Type / font scaling up to 200%

Keyboard navigation and focus rings full keyboard support: logical tab order that follows the visual layout, a visible 2px focus ring with 2px offset at 3:1 contrast via :focus-visible, Enter/Space activate, Escape closes overlays and returns focus to the trigger, arrow keys inside menus and lists, a "Skip to content" link, and focus trapped inside open modals

Screen reader labels screen reader support: semantic elements first (button, a, h1–h3, ul, label), an accessible name on every icon-only control (aria-label / SwiftUI .accessibilityLabel / Compose contentDescription), meaningful alt text or alt="" for decoration, headings for rotor navigation, status changes announced via aria-live or UIAccessibility.post, and related elements grouped so they read as one

Reduced motion respect reduced motion: when prefers-reduced-motion is set (SwiftUI @Environment(\.accessibilityReduceMotion), Android ANIMATOR_DURATION_SCALE = 0), swap slide/zoom/parallax transitions for a 150–200 ms cross-fade, stop auto-playing and looping animation, and keep essential feedback like progress indicators

Safe areas respect safe areas: backgrounds and media extend edge to edge, but text, buttons and tap targets stay inside the safe area insets (SwiftUI safe area / .ignoresSafeArea only for backgrounds, Android WindowInsets with enableEdgeToEdge, CSS env(safe-area-inset-top/bottom) with viewport-fit=cover); bottom buttons sit above the home indicator, never under it

Thumb zone design for one-handed use: primary actions, tab bar and search within the bottom third of the screen (bottom sheets and a bottom search bar rather than top-only controls), destructive or rare actions in the top corners, and nothing important only reachable in the top 25% of the screen on phones

Density two density modes: comfortable by default (48–56px rows, 16px cell padding, 14–16px text) and compact for data-heavy views (32–36px rows, 8px padding, 13px text), switchable in a view menu and remembered per user; touch layouts never drop below 44pt rows

Localization, RTL and text expansion localization-ready UI: no hard-coded strings, layouts that allow 40% longer text (buttons grow or wrap, never fixed widths), full RTL mirroring for Arabic and Hebrew using leading/trailing or CSS logical properties (margin-inline-start), directional icons flipped but not logos or media controls, and dates, numbers and currency via Intl / Locale formatters with ICU plurals

Container queries make components respond to their container, not the viewport: set container-type: inline-size on the wrapper, then @container (min-width: 400px) switches the card from stacked (image on top) to horizontal (image left, text, action right); size type and padding with container units (cqi); in Tailwind use @container on the parent and @md: variants on children; keep viewport media queries only for page-level layout

Native popover, dialog and anchor positioning use the platform for overlays instead of a JS positioning library: <button popovertarget="menu"> with <div id="menu" popover> for menus and popovers (top layer, light dismiss and Escape for free), <dialog> opened with showModal() for modals (::backdrop, inert page, focus moved and restored), and CSS anchor positioning (anchor-name on the trigger, position-anchor and position-area: bottom span-right on the panel, position-try-fallbacks: flip-block) so it flips near edges; add Floating UI only as a fallback where anchor positioning is unsupported

Fluid type with clamp() fluid typography with clamp(): headings like font-size: clamp(2rem, 1rem + 4vw, 4rem) so they scale smoothly from 32px at 400px wide to 64px at 1200px, body text clamp(1rem, 0.95rem + 0.25vw, 1.125rem); always mix a rem term into the preferred value so browser zoom and user font size still work; generate the scale with Utopia and apply the same approach to section spacing

Skip link and visually hidden text add a skip link as the first focusable element: <a href="#main" class="sr-only focus:not-sr-only"> "Skip to main content" that appears top-left with a strong focus ring on the first Tab and moves focus to <main id="main" tabindex="-1">; use visually hidden text (Tailwind sr-only: clipped, 1px, not display:none) to give icon buttons and repeated "Read more" links full names, e.g. "Delete invoice INV-204"

Apple Human Interface Guidelines follow the Apple Human Interface Guidelines using standard SwiftUI components (List with .insetGrouped style, Toggle, Picker with .segmented style, .borderedProminent buttons), SF Pro and SF Symbols, semantic system colors and Dynamic Type

Material Design 3 use Material Design 3 components (Jetpack Compose androidx.compose.material3, or MUI / @material/web on the web) with a color scheme generated from one seed color, the M3 type scale, and a filled, tonal, outlined button hierarchy

Microsoft Fluent 2 use Microsoft Fluent 2 components (@fluentui/react-components, Fluent UI React v9) inside a FluentProvider with webLightTheme / webDarkTheme, Segoe UI, Fluent System Icons, and MessageBar for inline status

IBM Carbon use IBM Carbon components (@carbon/react) with the White or Gray 100 theme, IBM Plex Sans, the UI Shell header, square corners, the 2x grid, and inline notifications for status

GitHub Primer use GitHub Primer components (@primer/react) with Octicons (@primer/octicons-react) and Primer's light and dark color tokens, UnderlineNav for tabs, Label and CounterLabel, and a green primary Button only for the main action on each screen

Shopify Polaris use Shopify Polaris components (Polaris web components, or @shopify/polaris in React) with Page, Card, IndexTable and Badge, the default Polaris tokens, and Shopify admin layout conventions

Radix UI and Radix Themes use Radix Themes (@radix-ui/themes) with an indigo accent and slate gray, the solid / soft / outline / ghost variants and Radix Colors scales; build anything custom on Radix Primitives (radix-ui) so it stays accessible

Tailwind UI / Tailwind Plus use Tailwind Plus (Tailwind UI) patterns and the Catalyst React kit with Tailwind CSS: indigo-600 primary, the gray scale, inputs with ring-1 ring-gray-300 and focus:ring-indigo-600, shadow-sm cards, Inter, and Headless UI for menus and dialogs

Ant Design use Ant Design components (antd v5) with ConfigProvider theme tokens, Table with pagination and Tag, Form in vertical layout, Alert and Breadcrumb, and @ant-design/icons; keep the density of Ant Design Pro